

Proactive Communication, Computing, and Caching (P3C): A Novel Method for MEC-Based 360° Mobile VR Video Streaming

Seoyun Park, Myoungjun Kim, Yeeun Ma, Dong Ho Kim, *Senior Member, IEEE*,
Cheolwoo You, *Member, IEEE*, and Jin-Hyun Ahn, *Member, IEEE*

Abstract—Mobile Virtual Reality Video (MVRV) is expected to be one of the most impactful applications in future wireless networks. However, it presents substantial challenges in satisfying the stringent latency and ultra-reliability requirements necessary for delivering a seamless and immersive user experience. To address these challenges, we propose a novel mobile edge computing (MEC)-based 360° MVRV streaming framework that incorporates proactive communication, computing, and caching (P3C). Unlike prior MEC-based approaches that limit the proactive stage to caching, the proposed P3C framework generalizes this stage to also include communication and computation. We design a complete P3C-based streaming system that integrates parallel process of communication and computing, aided by a deep neural network for dynamic performance optimization. In addition, we develop an analytical framework to achieve the end-to-end delay. Simulation results demonstrate that the proposed system substantially reduces delay compared to conventional methods.

Index Terms—Virtual reality, 360° video, field-of-view prediction, mobile edge computing, artificial intelligence

I. INTRODUCTION

Mobile virtual reality video (MVRV) [1], which refers to VR applications that stream content from a cloud server to lightweight user devices, has been recognized as a promising use case. More specifically, 360° MVRV, which integrates 360° video or three-degree-of-freedom (3-DoF) spherical video, has been extensively studied due to its immersive potential and commercial viability. However, 360° MVRV poses substantial technical challenges, including intensive demands on enhanced mobile broadband (eMBB) and stringent latency and reliability requirements associated with ultra-reliable low-latency communication (URLLC) services [2], [3].

To alleviate this, [4]–[6] have explored the method from the perspective of wireless communication. In [5], it has been

demonstrated that tile-based coding effectively reduces the bandwidth requirement by enabling to selectively delivers only the field-of-view (FoV) of the user’s head-mounted device (HMD) or prioritizes it with higher quality [5], [7]. Numerous studies [8]–[11] have employed field-of-view (FoV) communication based on tile-based coding to reduce bandwidth requirements. However, focusing solely on data transmission has not adequately addressed the stringent requirements posed by 360° MVRV, which operates without relying on high-end devices or locally stored content.

Recent studies have explored proactive caching strategies, wherein content is prefetched while the user is consuming the previous video chunk. As demonstrated in [1], [12], proactive caching effectively reduces end-to-end (E2E) delay. When integrated with FoV prediction [13], the enhancement is more significant. Consequently, proactive caching integrated with FoV prediction is regarded as a key technique for the practical deployment of 360° MVRV [14]. While it effectively reduces communication delays, the latency introduced by rendering high-volume 360° video remains a significant bottleneck.

To overcome this limitation, prior works [15]–[17] have proposed the integration of mobile edge computing (MEC) for computational offloading. In addition, it has been proposed to cache videos over two-hop link, exploiting the high-speed link between the cloud and the MEC. Among these efforts, [1] has provided a comprehensive architecture of MEC-based 360° MVRV streaming system. In this architecture, FoV is predicted and corresponding video tiles are cached to the MEC. After the FoV is achieved, hit tiles are rendered at the MEC or user’s head-mounted device (HMD). Finally, unhit tiles are sent to the user from the cloud and rendered at the user.

In this study, we propose a novel MEC-based 360° MVRV streaming system and an accompanying protocol that extends the architecture established in [1]. The key idea is proactive communication, computing, and caching (P3C) which generalize the proactive stage by enabling further tasks which are computing and communication. In conventional systems, the proactive stage is dedicated to caching the next set of video tiles from the cloud to the MEC server. However, our observations indicate that this strategy becomes suboptimal in the systems equipped with highly accurate AI-based FoV prediction. As the accuracy of the prediction network improves, the number of tiles required to achieve a satisfactory hit ratio is significantly reduced. Consequently, rather than allocating the entire proactive period solely to caching, the proposed P3C

S. Park and M. Kim contributed same. S. Park, M. Kim, Y. Ma, C. You, J.-H. Ahn are with Department of Information and Communication Engineering, Myongji University, Yongin, Gyeonggi-do 17058, Republic of Korea. (dhn04100@mju.ac.kr, tjmtjmj2237@mju.ac.kr, k41031614@mju.ac.kr, cwyou@mju.ac.kr, wlsus3396@mju.ac.kr) (Corresponding author: Jin-Hyun Ahn) D. H. Kim is with the IT Media Engineering Program, Seoul National University of Science and Technology, Seoul 01811, Republic of Korea (dongho.kim@seoultech.ac.kr).

This paper has supplementary downloadable material available at <http://ieeexplore.ieee.org>, provided by the author. The material includes protocols of proposed scheme. This material is 3.3 MB in size.

Manuscript received November 30, 2023; revised xx xx, xxxx; accepted xx xx, xxxx. Date of publication xx xx, xxxx. "This work was supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government(MSIT) (No.RS-2023-00229330, 3D Digital Media Streaming Service Technology)."

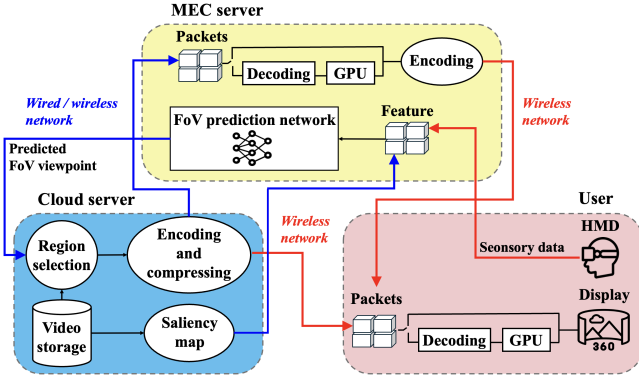


Fig. 1: Functional components of cloud, MEC, and user.

utilizes the proactive stage for more tasks, thereby reducing E2E delay significantly. In specific, the proposed system with P3C is incorporated with parallel processing of computing and communication which further enhances the performance, as will be shown in the simulation results.

We develop a deep neural network of cascaded structure, referred to as P3CNN, to decide three control variables and optimize the performance of proposed scheme for given streaming parameters. Furthermore, we derive an analytical framework of the delay with proposed system. The derived framework is used for the training of P3CNN and performance evaluation. We emphasize that the performance enhancement with the proposed system is surprisingly significant. To demonstrate the superiority, we compare the proposed scheme with various conventional schemes, including a MEC-disabled FoV streaming scheme, a MEC-disabled DASH [18] scheme, and a MEC-enabled caching-only scheme [1], [8], [19]–[21]. Furthermore, we provide various simulation results related to the robustness and an ablation study. The simulations are conducted using various 360° videos and a wide range of streaming environments to ensure comprehensive evaluation.

The remainder of this paper is organized as follows. Sec. II describes the system model upon which the proposed architecture is built. Sec. III introduces the concept of P3C and details the integration of P3C into the 360° MVRV streaming system. Sec. IV established the P3CNN. In Sec. V, we present simulation results and analyze the performance of the proposed system. Finally, conclusions are drawn in Sec. VI.

II. SYSTEM MODEL

Fig. 1 illustrates the functional components of the cloud, MEC, and user, where each entity is represented as a single component. As depicted, the cloud server encodes and compresses the requested video from the video storage. At the MEC server, FoV is predicted by the inference of pretrained FoV prediction network with the sensory data and saliency map as inputs. And the cached tiles at the MEC can be rendered or unrendered before being transmitted to the user. At the user's HMD, the unrendered tiles received from the cloud or MEC are rendered and the sensory data are collected for the FoV prediction. To simplify the analysis, detailed sub-tasks such as encoding, decoding, modulation, and compression are not explicitly modeled as separate delay components in the delay

TABLE I: Streaming parameters

Variable	Description
f	Number of frames per second
(N, M)	Number of tiles in a video frame
B	Data size per tile (bits)
F	Number of tiles in a FoV region
s	UDP packet size (bits)
t	Duration of a video chunk (s)
R_1	Transmission rate from cloud to MEC (bps)
R_2	Transmission rate from MEC to HMD (bps)
R_3	Transmission rate from cloud to HMD (bps)
W_M	Processing rate at the MEC (bps)
W_U	Processing rate at the HMD (bps)
h	Rate of size change after rendering
C_r	Compression ratio
D_{th}	E2E delay threshold (s)

analysis, as in [1]. Instead, their inherent computational load is implicitly encompassed within the broader assumption that time consumption of computing is directly proportional to the size of the data being transmitted or rendered, as in [22], [23]. The details on tile-based encoding, FoV prediction, rendering, and transmission are provided below. The notations introduced in this section are summarized in Table I.

A. Tile-based encoding

The video is partitioned into segments which are commonly referred to as *video chunks*. The video chunks are sequentially streamed to the user's HMD at each period. Each video chunk undergoes tile-based encoding, wherein only the tiles overlapping with the user's FoV are necessarily transmitted to the user. *Consequently, the set of FoV tiles is dynamically determined based on the video chunk and user, and even the number of FoV tiles, denoted by F , may vary.* Since the number of frames within one chunk is tf , the data size of FoV is $tfFB$. For ease of notation, we define a *tile* as the collection of all tiles across different frames within a video chunk.

B. FoV prediction

To support FoV prediction at the MEC server, the cloud server transmits a saliency map corresponding to the previous video chunks. Details regarding the construction and application of the saliency map are provided in [24]. In addition to this, the MEC server receives sensory data from the user's HMD, including head orientation and motion signals. By combining the saliency map with real-time sensory data, the MEC server predicts the user's FoV and communicates this prediction to the cloud server. Consequently, a prioritized list of tiles is generated, and a corresponding tile request is issued to the cloud. Then, the most probable tiles are cached to the MEC. We define the FoV hit ratio, denoted by ρ , as the proportion of hit tiles within the F FoV tiles, regardless of the number of cached tiles. We note that the FoV is assumed to remain invariant within each video chunk, following the approach in [1]. To ensure the validity of this assumption in practice, the chunk duration should be sufficiently short, or alternatively, the streaming region should be enlarged.

C. Rendering

The rendering process for 360-degree video is inherently more demanding than that of conventional 2D video, owing

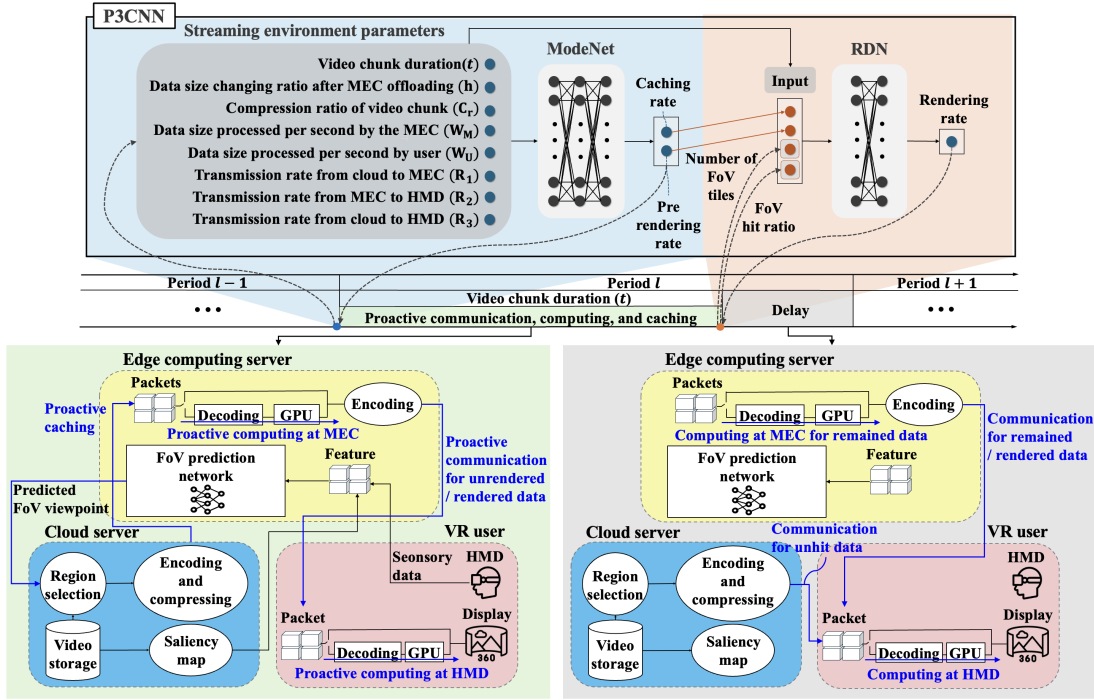


Fig. 2: Streaming system with P3C

to the need for spherical projection. For this reason, the computation can be offloaded to the MEC server. In particular, we adopt the partial offloading [25]. We denote the offloading ratio as r which means the ratio of data size to be rendered at the MEC server over the whole data size. While the time consuming for the rendering can be modeled more sophisticated considering tile-based encoding or viewpoint-dependent transformations, we model the computation time to be directly proportional to the size of the raw video data for simplicity, consistent with previous studies [1], [22], [23]. Correspondingly, the number of tiles which can be rendered during T sec at the MEC server can be modeled as $\lfloor TW_M/(tfB) \rfloor$. On the other hand, the necessary time to render n tiles is $tf n B / W_M$.

D. Transmission

Tiles are transmitted from the cloud to the MEC server for caching and are subsequently delivered to the HMD either in rendered form or as unrendered data. Tiles that are not cached at the MEC are fetched directly from the cloud and rendered on the HMD. We adopt a simplified packet protocol, assuming no overhead for ease of calculation. In particular, we consider a UDP-based transport layer, such as Quick UDP Internet Connections (QUIC) [26], [27]. Extension to other protocols, such as TCP-based mechanisms (e.g., 2TCP), which are commonly employed in DASH streaming [28], is straightforward. Leveraging the transmission rates rather than considering characteristics of physical layer, the number of tiles which can be cached during T sec can be modeled as $\lfloor sP_c/(tfBC_r) \rfloor$, where $P_c = \lfloor TR_1/s \rfloor$ is the number of transmitted packets. On the other hand, the necessary time to cache n tiles is sP_c/R_1 , where P_c is $\lceil ntfBC_r/s \rceil$.

III. PROACTIVE COMMUNICATION, COMPUTING, AND CACHING (P3C)

In contrast to conventional approaches, P3C allows the proactive operations throughout the entire streaming pipeline. Furthermore, the proposed system with P3C distinguishes itself from prior works by explicitly modeling the parallel execution of communication and computation processes. This section details the operational flow of the P3C, defines the eight kinds of P3C modes, and establishes an analytic framework to compute the delay with the proposed scheme.

A. Streaming system with P3C

1) *Proactive stage*: During period l , the user consumes a video chunk for a fixed duration t , as illustrated in Fig. 2. This fixed t marks the time dedicated to proactive operations while the user is consuming the current content. At the beginning of period l , the system captures the current streaming environment, characterized by $(t, h, C_r, W_M, W_U, R_1, R_2, R_3)$, and feeds it into P3CNN. The ModeNet, a subnetwork of P3CNN, then outputs the proactive P3C control variables (c, r_{pre}) . Here, c denotes caching rate and controls the proportion of t allocated to proactive caching, and r_{pre} denotes pre-rendering rate, controlling offloading at the MEC during this proactive stage.

Following the FoV prediction, encoding, and compression described in Sec. II, the cloud server transmits prioritized video tiles to the MEC server during initial ct s. During the following $(1-c)t$, proactive computing and communication of unrendered data to the HMD are firstly executed in a parallel manner at the MEC. Once the transmission of unrendered tiles completes, the HMD begins rendering the received tiles. If both proactive computing and communication is finished, the transmission of rendered data starts in parallel. This process is illustrated

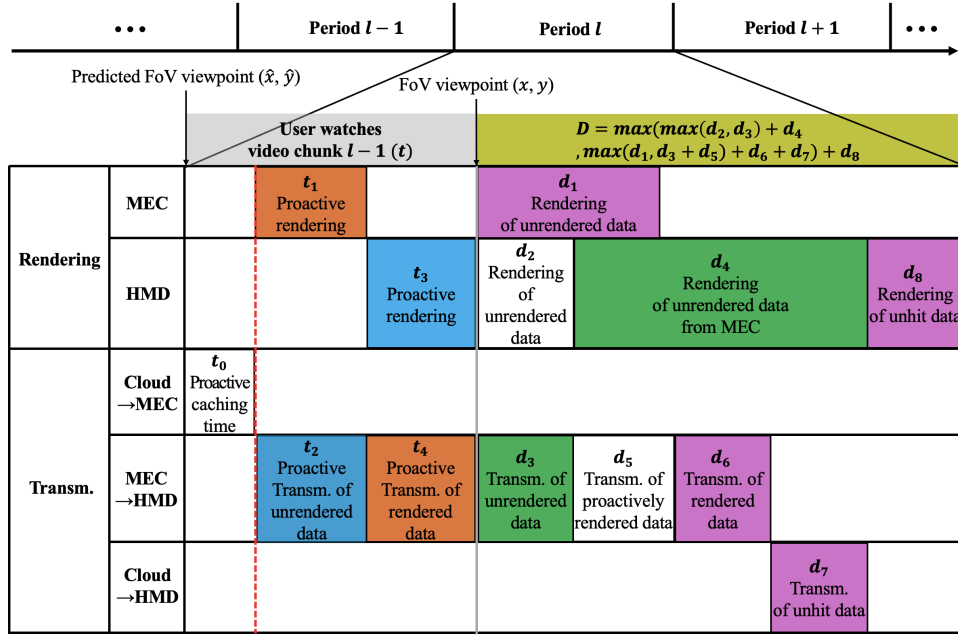


Fig. 3: P3C streaming timeline

TABLE II: Condition and associated delay of P3C modes.

Mode	Condition	Delay, D
1	$\max(\max(t_1, t_2) + t_4, t_2 + t_3) < (1 - c)t$	$d_7 + d_8$
2	$\max(t_1, t_2) + t_4 < (1 - c)t < t_2 + t_3$	$\max(d_2, d_7) + d_8$
3	$\max(t_1, t_2 + t_3) < (1 - c)t < \max(t_1, t_2) + t_4$	$d_5 + d_7 + d_8$
4	$\max(t_1, t_2) < (1 - c)t < \min(\max(t_1, t_2) + t_4, t_2 + t_3)$	$\max(d_2, d_5 + d_7) + d_8$
5	$t_2 + t_3 < (1 - c)t < t_1$	$\max(d_3 + d_4, \max(d_1, d_3 + d_5) + d_6 + d_7) + d_8$
6	$t_2 < (1 - c)t < \min(t_1, t_2 + t_3)$	$\max(\max(d_2, d_3) + d_4, \max(d_1, d_3 + d_5) + d_6 + d_7) + d_8$
7	$t_1 < (1 - c)t < t_2$	$\max(\max(d_2, d_3) + d_4, \max(d_1, d_3 + d_5) + d_6 + d_7) + d_8$
8	$(1 - c)t < \min(t_1, t_2)$	$\max(\max(d_2, d_3) + d_4, \max(d_1, d_3 + d_5) + d_6 + d_7) + d_8$

in Fig. 3. Each task in Fig. 3 is categorized according to its functional role and visually aligned in the figure based on these categories. Blocks sharing the same color follow a sequential execution, even if their categories differ.

The E2E delay is measured during the subsequent reactive stage, which commences immediately after this proactive period t concludes. This clear distinction is crucial. Period l identifies the entire cycle encompassing the user's consumption of video chunk l and the subsequent processing of video chunk $l + 1$. Within this cycle, t represents the fixed time for proactive preparation for chunk $l + 1$ while chunk l is being consumed. D , the E2E delay, then quantifies the variable latency that arises after t has finished, specifically for the communication and computing of video chunk $l + 1$ following a actual FoV request. This separation allows us to precisely evaluate the benefits of proactive operations by isolating the fixed content consumption time from the variable E2E delay.

Remark 1. The aforementioned description is an exemplary case, where the full set of proactive tasks are executed. In practice, however, it may terminate before the tasks are completed or even start. Consequently, various configurations of the proactive stage arise, which we categorize into eight distinct cases, referred to as P3C modes, as discussed later. It is also worth noting that both the configuration and its

associated performance depend on the control variables, which are determined by the ModeNet.

Remark 2. The P3C framework operates based on prediction, and therefore, not all cached tiles can perfectly satisfy the user's actual FoV requirements. Some cached tiles are rendered or transmitted to the user without a guarantee of being hit, and the unhit tiles are discarded, which may appear inefficient. Although, we assert that the benefit gained from optimizing the user-watching time outweighs the potential waste. Here, optimization of the user-watching time refers to utilizing the available time not only for caching, but also for computing and communication. The simulation results support and validate this claim.

2) *Reactive stage:* Once the proactive stage t concludes, the user issues a FoV request for the subsequent video chunk. At this precise moment, which marks the transition from the fixed proactive period t , the user's actual FoV for the next chunk is definitively known. From this point onward, the system's objective is to ensure that all required FoV tiles for this actual FoV are fully rendered at the user side. This entire interval, from the precise moment of the FoV request until the final rendering is complete at the user's HMD, constitutes the E2E delay. This E2E delay, denoted as D , represents the variable delay that our

TABLE III: Explicit expression of N_i , $i = 1, \dots, 4$ for P3C modes.

Mode	N_1	N_2	N_3	N_4
1	0	0	N_c	0
2	0	0	$\lfloor r_{\text{pre}} N_c \rfloor + \left\lfloor \frac{((1-c)t-t_2)W_U}{tfB} \right\rfloor$	$N_c - N_3$
3	$\lfloor r_{\text{pre}} N_c \rfloor - \left\lfloor \frac{((1-c)t-\max(t_1, t_2))R_2}{s} \right\rfloor \left\lfloor \frac{s}{tfBhC_r} \right\rfloor$	0	$N_c - N_1$	0
4	$\lfloor r_{\text{pre}} N_c \rfloor - \left\lfloor \frac{((1-c)t-\max(t_1, t_2))R_2}{s} \right\rfloor \left\lfloor \frac{s}{tfBhC_r} \right\rfloor$	0	$N_c - N_1 - N_4$	$N_c - \lfloor r_{\text{pre}} N_c \rfloor - \left\lfloor \frac{((1-c)t-t_2)W_U}{tfB} \right\rfloor$
5	$\left\lfloor \frac{(1-c)t \cdot W_M}{tfB} \right\rfloor$	$\lfloor r_{\text{pre}} N_c \rfloor - N_1$	$N_c - \lfloor r_{\text{pre}} N_c \rfloor$	0
6	$\left\lfloor \frac{(1-c)t \cdot W_M}{tfB} \right\rfloor$	$\lfloor r_{\text{pre}} N_c \rfloor - N_1$	$\left\lfloor \frac{((1-c)t-t_2)W_U}{tfB} \right\rfloor$	$N_c - \lfloor r_{\text{pre}} N_c \rfloor - N_3$
7	$\lfloor r_{\text{pre}} N_c \rfloor$	$N_c - N_1 - N_4$	0	$\left\lfloor \frac{(1-c)t \cdot R_2}{s} \right\rfloor \cdot \left\lfloor \frac{s}{tfBC_r} \right\rfloor$
8	$\left\lfloor \frac{(1-c)t \cdot W_M}{tfB} \right\rfloor$	$N_c - N_1 - N_4$	0	$\left\lfloor \frac{(1-c)t \cdot R_2}{s} \right\rfloor \cdot \left\lfloor \frac{s}{tfBC_r} \right\rfloor$

P3C framework is explicitly designed to minimize, ensuring a seamless and immersive user experience. Its operational flow is carefully orchestrated. As an initial step, only the *hit tiles* among the distributed cached tiles are retained. *We remark that the cached tiles are categorized into one of four states, which are rendered tiles at the MEC, unrendered tiles at the MEC, rendered tiles at the HMD, or unrendered tiles at the HMD.* As depicted in Fig. 2, the rendering decision network (RDN), a subnetwork within the P3CNN framework, then infers the reactive control variable *rendering rate* r which determines the offloading ratio at the MEC. The decision for r is made dynamically at the start of the reactive stage, leveraging the newly available actual FoV information and the current system parameters. As illustrated in Fig. 3, the computation of unrendered data and the transmission of unrendered data are executed in parallel at the MEC server, while computation of unrendered data is simultaneously carried out at the HMD. Subsequently, the MEC performs sequential transmission of proactively rendered data followed by rendered data. Notably, the transmission of rendered data must begin only after the MEC completes the computation of unrendered data. At the HMD, the computation of unrendered data received from the MEC starts after the corresponding transmission is completed. Finally, the communication and computation of unhit data are executed after the MEC-to-HMD communication concludes.

Remark 3. *The described reactive stage represents an exemplary scenario in which all associated tasks are executed. For instance, if all necessary tasks are successfully completed during the proactive stage, the reactive stage may be reduced to merely transmitting and rendering unhit tiles. In addition, we remark that the proposed P3C framework deliberately excludes pipelining between communication and rendering. It is because implementing pipelining would introduce serious technical challenges, including increased protocol complexity, difficulties in modeling E2E delay, and hardware issues such as precise synchronization.*

B. P3C modes

While Sec. III-A describes the full set of tasks executed during the proactive and reactive stages, the feasible pairs

of proactive stage and reactive stage are categorized into eight cases, each referred to as a P3C mode. The distinct configurations of P3C modes can be found in the supplementary material. We introduce the conditions and associated delays of P3C modes in TABLE II. As depicted in Fig. 3, $t_0, \dots, t_5$ are the durations of tasks in the proactive stage and $d_1, \dots, d_8$ are the durations of tasks in the reactive stage.

C. Analytic framework of E2E delay

We now establish the analytic framework for obtaining E2E delay by formulating the condition and associated delay in TABLE II into a function of streaming parameters, control variables, and FoV-related variables.

Theorem 1. *The conditions of P3C modes are expressed by $t_0, \dots, t_4$ in Table II, where those are formulated into*

$$t_0 = ct, \quad (1)$$

$$t_1 = \lfloor r_{\text{pre}} N_c \rfloor \cdot \frac{tfB}{W_M}, \quad (2)$$

$$t_2 = \left\lceil \frac{(N_c - \lfloor r_{\text{pre}} N_c \rfloor) \cdot tfBC_r}{s} \right\rceil \cdot \frac{s}{R_2} \quad (3)$$

$$t_3 = (N_c - \lfloor r_{\text{pre}} N_c \rfloor) \cdot \frac{tfB}{W_U}, \quad (4)$$

$$t_4 = \left\lceil \frac{\lfloor r_{\text{pre}} N_c \rfloor \cdot tfBC_r h}{s} \right\rceil \cdot \frac{s}{R_2}, \quad (5)$$

and

$$N_c = \left\lfloor \left\lfloor \frac{t_0 \cdot R_1}{s} \right\rfloor \cdot \frac{s}{tfBC_r} \right\rfloor, \quad (6)$$

based on the system model introduced in Sec. II. N_c denotes the number of cached tiles.

Proof: We omit the proof, as it can be directly derived based on the system model presented in Sec. II. ■

We now provide the analytical formulation of E2E delay. It is worthwhile to note that the formulation depends on $\Gamma_1, \dots, \Gamma_4$ which denote the number of hit tiles among the cached tiles distributed across four categories.

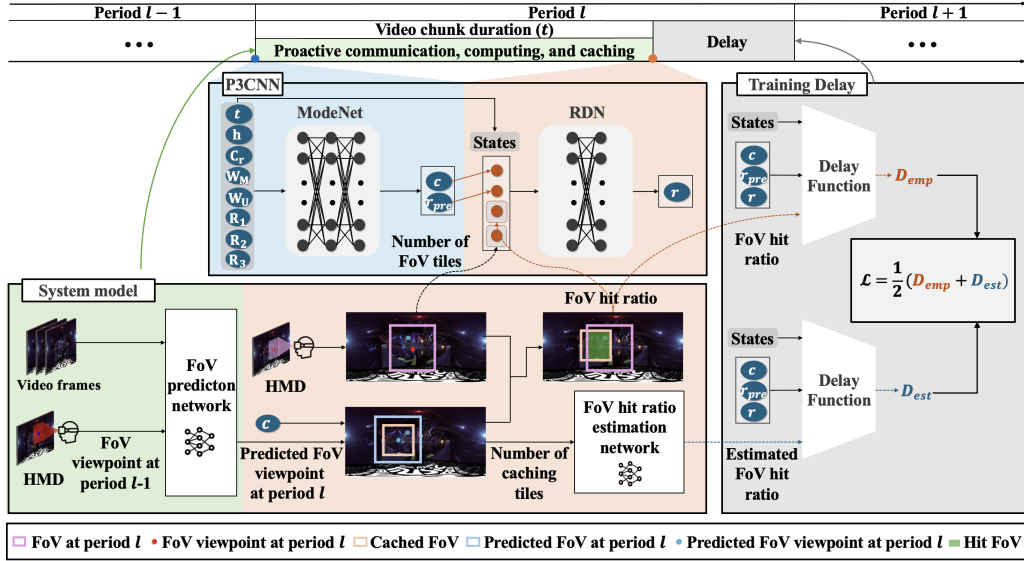


Fig. 4: Workflow of P3CNN training.

Theorem 2. The E2E delay in the proposed streaming system with P3C can be formulated based on the system model introduced in Sec. II, as

$$\max(\max(d_2, d_3) + d_4, \max(d_1, d_3 + d_5) + d_6 + d_7) + d_8, \quad (7)$$

where

$$d_1 = \lfloor r\Gamma_2 \rfloor \cdot \frac{tfB}{W_M}, \quad (8)$$

$$d_2 = \Gamma_4 \cdot \frac{tfB}{W_U}, \quad (9)$$

$$d_3 = \left\lceil (\Gamma_2 - \lfloor r\Gamma_2 \rfloor) \frac{tfBC_r}{s} \right\rceil \cdot \frac{s}{R_2}, \quad (10)$$

$$d_4 = (\Gamma_2 - \lfloor r\Gamma_2 \rfloor) \cdot \frac{tfB}{W_U}, \quad (11)$$

$$d_5 = \left\lceil \frac{\Gamma_1 tfBhC_r}{s} \right\rceil \cdot \frac{s}{R_2}, \quad (12)$$

$$d_6 = \left\lceil \frac{\lfloor r\Gamma_2 \rfloor \cdot tfBhC_r}{s} \right\rceil \cdot \frac{s}{R_2}, \quad (13)$$

$$d_7 = \left\lceil \frac{(F - \Gamma_1 - \Gamma_2 - \Gamma_3 - \Gamma_4) tfBC_r}{s} \right\rceil \cdot \frac{s}{R_3}, \quad (14)$$

$$d_8 = (F - \Gamma_1 - \Gamma_2 - \Gamma_3 - \Gamma_4) \cdot \frac{tfB}{W_U}. \quad (15)$$

Herein, $\Gamma_1, \dots, \Gamma_4$ denote the numbers of hit tiles among the rendered tiles at the MEC, unrendered tiles at the MEC, rendered tiles at the HMD, and unrendered tiles at the HMD, respectively, after the proactive stage.

Proof: We omit the proof, as it can be directly derived based on the system model presented in Sec. II. ■

Remark 4. While Theorem 2 provides a complete formulation for computing the E2E delay, it does not capture the influence of the proactive stage, as the quantities $\Gamma_1, \dots, \Gamma_4$ are independent of the proactive operations. However, to develop a

delay model that accurately captures the impact of streaming parameters and control variables, we express $\Gamma_1, \dots, \Gamma_4$ as

$$\Gamma_i = \rho_i N_i, \quad \text{for } i = 1, \dots, 4, \quad (16)$$

where N_i is the number of cached tiles at each category, and ρ_i is the ratio of the number of hit tiles to N_i . Considering that ρ_i is given as constant, We derive N_i as a function of streaming parameters and control variables. We remark that these formulations vary across different P3C modes. The complete mode-specific expressions of N_i , $i = 1, \dots, 4$, are provided in TABLE III. The derivation is omitted since it can be easily derived based on the system model presented in Sec. II.

IV. P3CNN

We now elaborate on the design and training of P3CNN. The objective of P3CNN is to optimize the performance of the P3C framework by determining optimal control variables for a given streaming environment, which includes specific attributes of the video chunk and the user. Accordingly, it can be expressed as

$$\underset{\theta, \phi}{\operatorname{argmin}} \mathbb{E}_{\mathbf{s} \in S, (v, u) \in V \times U} [D], \quad (17)$$

$$c, r_{\text{pre}} = M_{\theta}(\mathbf{s}), \quad (18)$$

$$r = R_{\phi}(\mathbf{s}, c, r_{\text{pre}}, \rho_1, \rho_2, \rho_3, \rho_4), \quad (19)$$

$$\mathbf{s} = (t, h, C_r, W_M, W_U, R_1, R_2, R_3). \quad (20)$$

Herein, V and U denote the training datasets, each representing a collection of video chunks and users, respectively. We denote the ModeNet by $M_{\theta}(\cdot)$ and the RDN by $R_{\phi}(\cdot)$, where θ and ϕ are the trainable parameters of each network. The E2E delay D can be analytically computed using the framework established in Theorem 2, Remark 4, and Table III.

However, training P3CNN with only E2E delay D as the loss function has been observed to result in slow convergence and divergence of the training loss, as demonstrated in the simulation result. To alleviate this, we reduce the number of

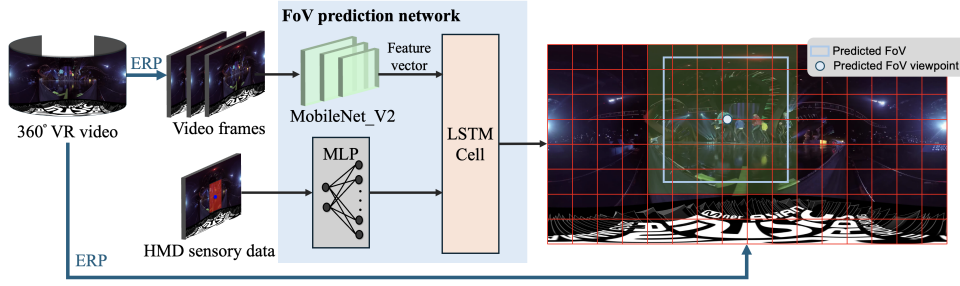


Fig. 5: ERP and FoV prediction network.

trainable parameters by introducing the following simplifying relation of

$$\Gamma_i = \frac{\rho F}{N_c} N_i, \quad \text{for } i = 1, \dots, 4, \quad (21)$$

in the calculation of delay for loss, where ρ is the FoV hit ratio defined in Sec. II-B. It is a direct result of assumption that the hit tiles are uniformly distributed across the four cached categories. Given that the total number of hit tiles is ρF , the hit ratio for each category is $\rho F/N_c$. This uniform distribution assumption for hit tiles is applied solely during the P3CNN training phase to enhance learning efficiency and convergence stability. It is crucial to note that for the final system performance evaluation in Sec. V, this uniform distribution assumption is no longer applied. Instead, actual hit patterns derived from FoV hit ratio are rigorously utilized to determine $\Gamma_1, \dots, \Gamma_4$.

In addition, we set F as the input of RDN to allow the model to utilize the newly achieved information, where F is dynamically determined. Accordingly, the RDN formulation in (19) is updated to $r = R_\phi(s, c, r_{\text{pre}}, \rho, F)$.

Additionally, we observe that modeling N_c and ρ independently fails to capture their generally proportional relationship. We alternatively define the training loss, $\mathcal{L}$, as the average of two complementary delay, which are the empirical delay, D_{emp} , and the estimated delay, D_{est} . Correspondingly, the loss, $\mathcal{L}$ is expressed as

$$\mathcal{L} = \frac{1}{2}(D_{\text{emp}} + D_{\text{est}}). \quad (22)$$

The empirical delay remains consistent with the actual delay, D , as it uses the actual FoV hit ratio, ρ . In contrast, we employ a pretrained network for estimating the FoV hit ratio used in computing D_{est} . The FoV hit ratio estimation network models the FoV hit ratio with a given caching size. It is trained across diverse users and video content. In this case, the FoV hit ratio is $E_\Psi(N_c)$, where $E_\Psi(\cdot)$ is the estimation network, thereby enabling the caching size and related streaming parameters to be learned with respect to the FoV hit ratio. The effectiveness of the proposed loss function is validated through simulation results. The overall training procedure of P3CNN is illustrated in Fig. 4.

V. EXPERIMENTS

In this section, we evaluate the performance of the proposed P3C system and compare it with conventional benchmark schemes. To this end, we first present the simulation setup,

including detailed configurations of the P3C system and the baseline methods. Subsequently, we provide numerical results.

A. Simulation setup

1) *Datasets*: We employ the panoramic video sequences with head movement and eye movement (PVS-HMEM) introduced in [13], which comprises 76 panoramic video clips collected from YouTube. Each clip includes head and eye movement data from 55 users. The videos are presented in a 2D equirectangular projection (ERP) with a resolution of 3840×1920 pixels at 60 frames per second. The user's FoV is assumed to cover 1080×1200 pixels. Each video frame is partitioned into a 16×8 grid, yielding 128 tiles in total. Following the methodology in [29], [30], FoV tiles are defined as the set of tiles that spatially overlap with the FoV region. The number of FoV tiles varies among 25, 30, and 36.

2) *Streaming environments*: The streaming environment used in our experiments emulates a realistic MEC-based 360° MVRV streaming. We consider the asymmetry between resource-constrained HMDs such as Oculus Quest 2/3 and high-performance MEC servers. Specifically, the MEC server is assumed to be equipped with high-end hardware, including GPUs such as NVIDIA A100 or T4 and CPUs like Intel Xeon or AMD EPYC. In contrast, HMDs are limited by power and thermal constraints, resulting in significantly lower processing power. To reflect this computational disparity, the processing rate of the MEC server, W_M , is defined as a multiple of the user's processing rate, W_U . The ratio is chosen range from $10\times$ to $100\times$ faster. We consider realistic transmission environments including cloud-to-MEC backbones and wireless MEC-to-HMD links such as Wi-Fi 6 or 5G. Transmission rate from the cloud to the user, R_3 , serve as the base range, and rate from cloud to MEC, R_1 , and MEC to HMD, R_2 , are expressed as proportional factors of R_3 . All key parameters are defined over logarithmic scales to comprehensively capture both extreme and average conditions. Furthermore, the E2E delay for performance evaluation in our experiments is computed based on Theorem 2. Table IV summarizes the simulation defaults and parameter ranges. We note that the system parameters may vary over time, reflecting the dynamic nature of practical network environments.

3) *FoV prediction*: In this work, we consider a simpler method that uses raw 2D frames as input and predict the FoV center coordinate, rather than utilizing saliency map. Correspondingly, the MEC server received 2D frames which

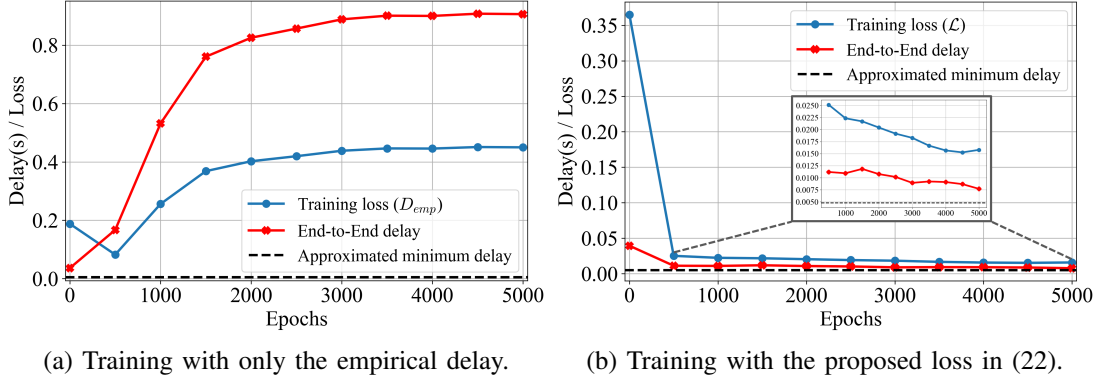


Fig. 6: Training of P3CNN.

TABLE IV: Simulation parameters.

Variable	Range or Value
f	60 fps
(N, M)	(16, 8)
B	20 Mbit
F	[25, 36] (e.g., 25, 30, 36)
s	1460 bytes
t	[0.5, 1.0] s
R_1	$[10^{0.3010}, 10^1] \times R_3$ bps
R_2	$[10^{0.3010}, 10^1] \times R_3$ bps
R_3	$[10^8, 10^9]$ bps
W_M	$[10^1, 10^2] \times W_U$ bps
W_U	$[10^{9.505}, 10^{11}]$ bps
h	[1.5, 5.0]
C_r	[1/600, 1/400]
D_{th}	0.02 s

are mapped to 2D ERP, from the cloud server, along with past head motion and gaze data collected from the user's HMD. Based on this input, it predicts the center coordinates of the user's future FoV. Using the predicted coordinates, the server can generate a list of nearby tiles to be proactively cached. This tile prioritization is a critical aspect of our P3C framework, applied consistently throughout both the P3CNN training and simulation phases. Tiles are always prioritized based on their proximity to the predicted FoV center for allocation to different processing and caching categories. Tiles closer to the predicted FoV center are preferentially assigned to categories that lead to lower E2E delay, specifically in the order of user rendering first, followed by MEC rendering, then user unrendering, and finally MEC unrendering. We remark that the hit tiles are assumed to be uniformly distributed, computing the D_{emp} and D_{est} for the simplicity of training in contrast. As thoroughly detailed in Sec. IV, this uniform distribution assumption for hit tiles applies solely during the P3CNN training phase.

Fig. 5 illustrates the ERP and prediction network used in the simulations. The prediction network takes the past 30 video frames and corresponding HMD sensory data as input. It consists of a MobileNet_V2-based CNN [31], a multilayer perceptron (MLP) network, and a LSTM network [32]. Out of the 76 video clips in the PVS-HMEM dataset, 50 clips are used for training of FoV prediction network, 13 are allocated for training the P3CNN, and the other 13 are reserved for testing.

The FoV prediction network is trained using a loss function defined as the Euclidean distance between the predicted and actual FoV center coordinates. We use the Adam optimizer with a learning rate of 0.0001 and a batch size of 16. While the performance varies across the test clips, the average accuracy is 73.78%. The accuracy is defined as the overlapping ratio between the predicted and actual FoV tiles. For the final system performance evaluation, the uniform distribution assumption for hit tiles is discarded. Instead, actual hit tiles for each of the four categories ($\Gamma_1, \Gamma_2, \Gamma_3, \Gamma_4$) are precisely determined by comparing the actual FoV with the specific allocated tiles (N_1, N_2, N_3, N_4), thereby reflecting non-uniform and dynamic distribution of hit tiles.

4) *P3CNN*: Both subnetworks of P3CNN follow a compact feedforward structure with linear layers and ReLU activations. ModeNet consists of 9 hidden layers, while RDN consists of 5 hidden layers. Each data for training is defined as the combination of streaming parameters randomly determined in the Table IV and a pair of video chunk and user. The video chunks and users are randomly chosen among the 13 video clips and 55 users in the PVS-HMEM. P3CNN is trained using the Adam optimizer with a learning rate of 5×10^{-6} for 5000 epochs. We apply gradient accumulation [33], [34], which accelerates the training. While the delay model is originally established by discretizing data into tile and packet units, we relax this constraint during training by allowing fractional values for tiles and packets. This enables the application of gradient-based optimization methods such as gradient descent.

Fig.6 presents the training results when using either the empirical delay D_{emp} alone or the proposed loss function in (22). The "approximated minimum delay" refers to the optimal E2E delay obtained by exhaustively searching over discretized values of the control variables (c, r_{pre}, r), each divided into 100 grid points. When training with only D_{emp} as the loss, both the training loss and the E2E delay exhibit divergence after a certain number of epochs. In contrast, when employing the proposed loss $\mathcal{L}$, both metrics demonstrate steady convergence toward the optimal baseline. These results confirm that the proposed loss function $\mathcal{L}$ is effective and reliable for training the P3CNN.

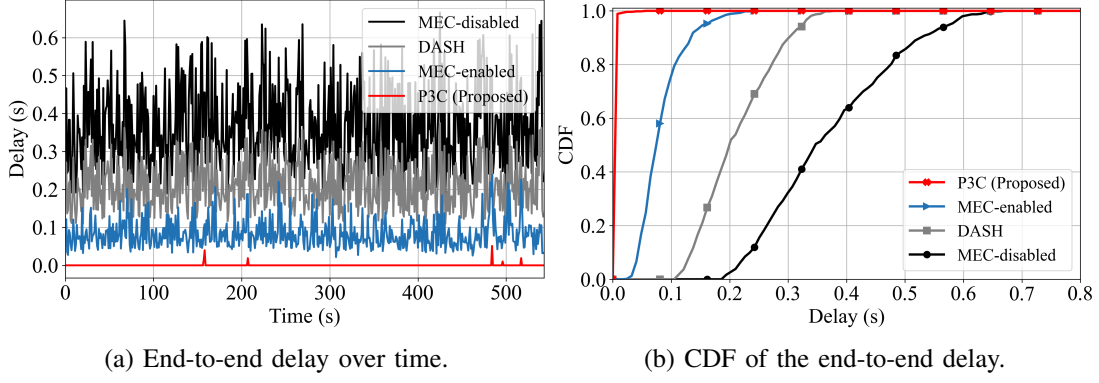
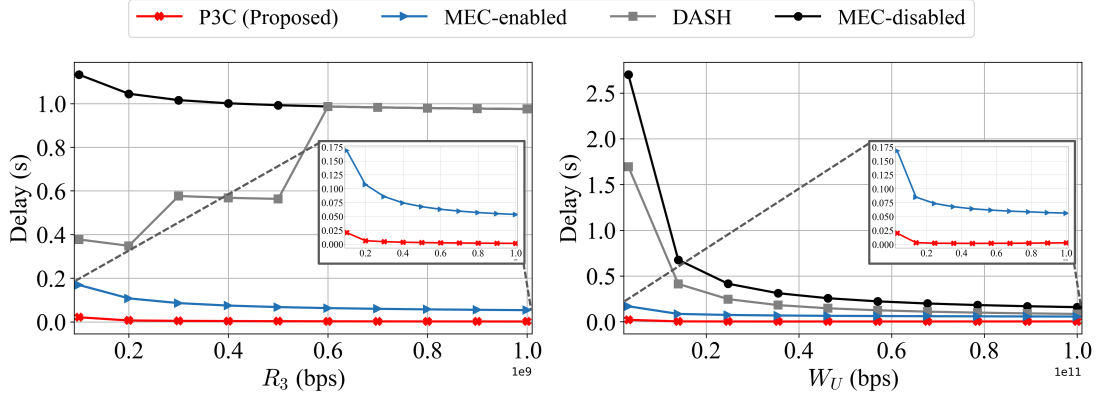


Fig. 7: End-to-end delay for a concatenated test video.

Fig. 8: Delay sensitivity with respect to R_3 (left) and W_U (right).

B. Benchmark schemes

1) *MEC-disabled*: In this baseline scheme, the MEC infrastructure is not utilized, and no tasks are performed during the proactive stage. Once the user finishes watching the current video chunk, a request for the next set of FoV tiles is sent directly to the cloud. These tiles must then be transmitted and rendered in real time.

2) *DASH*: The conventional DASH is adopted for 360° MVRV streaming. Similar to the “MEC-disabled”, the MEC is not utilized, but the tile-based encoding is leveraged. The quality of each tile is adaptively selected based on the downlink bandwidth R_3 . Three resolution levels are supported, which are 1080p, 1440p, and 1920p. The original video has a resolution of 3840×1920 , and each 1920p tile corresponds to a 240×240 pixel region. The 1440p and 1080p tiles correspond to approximately 56.3% and 31.6% of the 1920p tile size, respectively. The resolution of each tile is determined as 1080p for $R_3 < 3 \times 10^8$, 1440p for $3 \times 10^8 \leq R_3 < 6 \times 10^8$, and 1920p for $6 \times 10^8 \leq R_3$. Given the selected resolution, the delay is computed to be the time to transmit and render the FoV tiles leveraging only the link between the cloud and user. The computation follows the system model in Sec. II.

3) *MEC-enabled*: This scheme refers to the MEC-based architecture proposed in [1]. In contrast to the proposed scheme, this baseline supports only proactive caching and does not implement parallel processing of computation and

communication. Although the rendering rate can theoretically vary over the range $[0, 1]$, either 0 or 1 yields the best performance. In particular, the proposed architecture in [1] operates in a fully sequential manner and does not allow parallel computation between the MEC and the HMD. While the hit tiles at the MEC are (i) rendered and sent, or alternatively, (ii) sent and rendered, one of these options will always be faster depending on the current network and computation conditions. Accordingly, it fully relies on either the MEC or the user for rendering to optimize the performance.

C. Comparison results

In Fig. 7, the E2E delay for each video chunk is measured and depicted in two forms, which are delay over time and cumulative distribution function (CDF). This evaluation is conducted using a concatenated sequence of 13 video clips streamed to a specific user. For the streaming parameters, the value at streaming of each video chunk are set as $t = 0.5s$, $h = 3.5$, $C_r = 1/400$, $W_M \in [10^1, 10^{1.5}]$, $W_U \in [10^{10.0}, 10^{10.5}]$ bps, $R_1, R_2 \in [10^{0.35}, 10^{0.75}]$, and $R_3 \in [10^8, 10^{8.6}]$ bps. This setup reflects the time-varying transmission rates and processing rates. In Fig. 7, it is shown that the “P3C (Proposed)” significantly outperforms the baseline schemes. In specific, the “MEC-disabled” and “DASH” exhibit significantly higher delay values with frequent and large delay spikes. Even the “MEC-enabled”, despite leveraging edge

Parameters	R_3 (10^8 bps)					W_U (10^9 bps)				
	1.0	3.0	5.0	7.0	9.0	3.2	24.7	46.2	67.7	89.2
P3C (Proposed)	0.0209 s	0.0045 s	0.0029 s	0.0021 s	0.0017 s	0.0202 s	0.0022 s	0.0019 s	0.0021 s	0.0026 s
MEC-enabled	0.1688 s	0.1075 s	0.0676 s	0.0596 s	0.0551 s	2.7005 s	0.4147 s	0.2565 s	0.1988 s	0.1689 s
DASH	0.3774 s	0.5765 s	0.5632 s	0.9825 s	0.9769 s	1.6950 s	0.2466 s	0.1464 s	0.1098 s	0.0909 s
MEC-disabled	1.1325 s	1.0158 s	0.9925 s	0.9825 s	0.9769 s	2.7005 s	0.4147 s	0.2565 s	0.1988 s	0.1689 s
Parameters	t (s)					h				
	0.5	0.611	0.722	0.833	0.944	1.5	2.278	3.056	3.833	4.611
P3C (Proposed)	0.0050 s	0.0059 s	0.0069 s	0.0083 s	0.0108 s	0.0030 s	0.0043 s	0.0061 s	0.0084 s	0.0123 s
MEC-enabled	0.0639 s	0.0782 s	0.0923 s	0.1065 s	0.1207 s	0.0674 s	0.0812 s	0.0939 s	0.1058 s	0.1174 s
DASH	0.4253 s	0.5198 s	0.6143 s	0.7088 s	0.8033 s	0.6380 s	0.6380 s	0.6380 s	0.6380 s	0.6380 s
MEC-disabled	0.6882 s	0.8411 s	0.9940 s	1.1470 s	1.2999 s	1.0323 s	1.0323 s	1.0323 s	1.0323 s	1.0323 s
Parameters	C_r (10^{-3})					$W_M (\times 10 \cdot W_U)$				
	1.667	1.852	2.037	2.222	2.407	1	3	5	7	9
P3C (Proposed)	0.0057 s	0.0071 s	0.0081 s	0.0104 s	0.0126 s	0.0178 s	0.0055 s	0.0041 s	0.0035 s	0.0032 s
MEC-enabled	0.0850 s	0.0894 s	0.0943 s	0.0990 s	0.1034 s	1.0323 s	1.0323 s	1.0323 s	1.0323 s	1.0323 s
DASH	0.6685 s	0.6713 s	0.6742 s	0.6771 s	0.6800 s	0.6380 s	0.6380 s	0.6380 s	0.6380 s	0.6380 s
MEC-disabled	1.0900 s	1.0971 s	1.1042 s	1.1113 s	1.1185 s	1.0323 s	1.0323 s	1.0323 s	1.0323 s	1.0323 s
Parameters	$R_1 (\times R_3)$					$R_2 (\times R_3)$				
	2.0	3.8	5.6	7.3	9.1	2.0	3.8	5.6	7.3	9.1
P3C (Proposed)	0.0144 s	0.0056 s	0.0053 s	0.0053 s	0.0054 s	0.0185 s	0.0083 s	0.0045 s	0.0033 s	0.0026 s
MEC-enabled	0.0958 s	0.0958 s	0.0958 s	0.0958 s	0.0958 s	0.1464 s	0.0990 s	0.0810 s	0.0713 s	0.0652 s
DASH	0.6380 s	0.6380 s	0.6380 s	0.6380 s	0.6380 s	0.6380 s	0.6380 s	0.6380 s	0.6380 s	0.6380 s
MEC-disabled	1.0323 s	1.0323 s	1.0323 s	1.0323 s	1.0323 s	1.0323 s	1.0323 s	1.0323 s	1.0323 s	1.0323 s

TABLE V: Comparison of E2E delay across varying system parameters.

computing, also shows substantial delay. In specific, the outage probability for $D_{th} = 0.02$ s of the proposed scheme is 0.37%, where $D_{th} = 0.02$ s aligns with the perceptual delay threshold discussed in [35]. The outage probabilities of other schemes are equivalently 100%.

Next, we consider a wide range of streaming parameters as described in TABLE IV. For the eight streaming parameters, we fix one while systematically varying the others within their respective ranges. In particular, the E2E delay corresponding to each fixed value is computed as the average E2E delay over all combinations of the other parameters. Each range of parameter is discretized into four logarithmically uniform intervals, and the resulting values are used for the evaluation. Fig. 8 presents the comparative results, where either R_3 or W_U is fixed. The complete set of results is summarized in Table V.

In Fig. 8 (left), all schemes except “DASH” show relatively stable delay, which decreases as R_3 increases. The proposed scheme consistently achieves the lowest delay across all levels of transmission rate. In contrast, “DASH” exhibits increase like a staircase in delay due to its bitrate adaptation mechanism. In Fig. 8 (right), as W_U increases, all methods show a general trend of reduced delay. The proposed scheme maintains the lowest delay regardless of the user’s computational capacity. It is shown that the “MEC-disabled” and “DASH” are significantly sensitive to W_U because those entirely rely on the user’s computing power. Table V confirms that the proposed scheme maintains robustness across diverse streaming environments.

D. Discussion

We present additional analyses to further validate the effectiveness of the proposed P3C. First, we analyze how

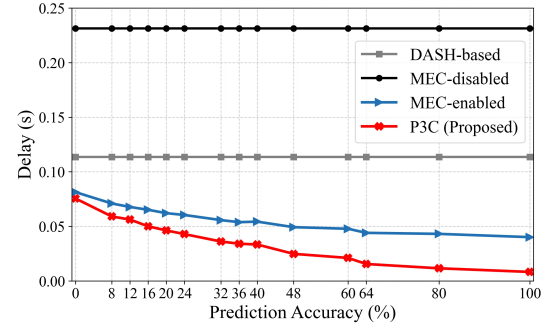


Fig. 9: Delay according to FoV prediction accuracy.

the FoV prediction accuracy affects the E2E delay. Next, we provide an ablation study to discuss the role of several modules in terms of performance improvement. Finally, we examine where the primary bottlenecks of E2E delay occur and investigate how the proposed method efficiently utilizes system resources under various environments.

1) The effect of FoV prediction accuracy on E2E delay:

Fig. 9 shows the variation in E2E delay according to the prediction accuracy. Here, the prediction accuracy does not represent the hit ratio of cached data, but rather the performance of FoV prediction network, which is the degree of overlap between the FoV tiles predicted by the model and the actual user’s FoV tiles. For experimental setup, the range of R_3 is reduced to $[10^6, 3 \times 10^8]$ bps, compared to the value in TABLE. IV for a clearer explanation. When the prediction accuracy increases, more tiles among the FoV tiles are cached at the MEC server or HMD, resulting in a reduction in delay.

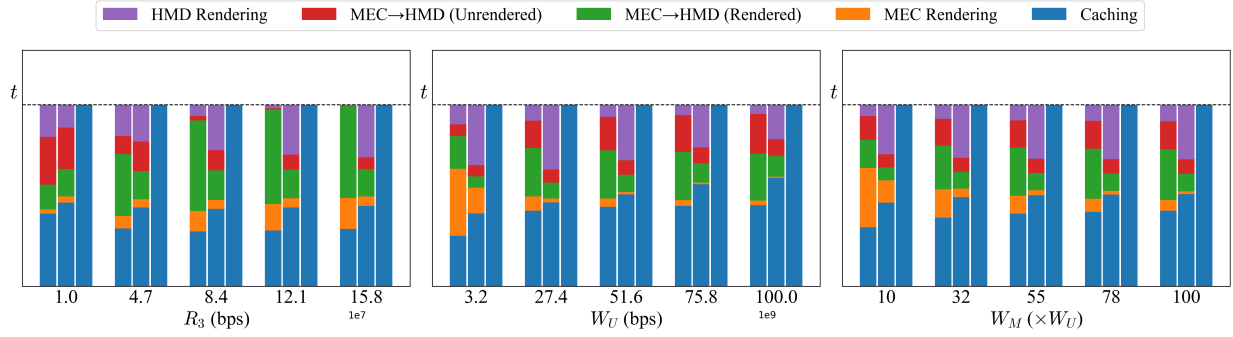


Fig. 10: Composition ratio during the proactive stage with respect to R_3 , W_U , and W_M . (Each group of bars corresponds to different schemes: Proposed, Proposed w/o P3CNN, and MEC-enabled in that order.)

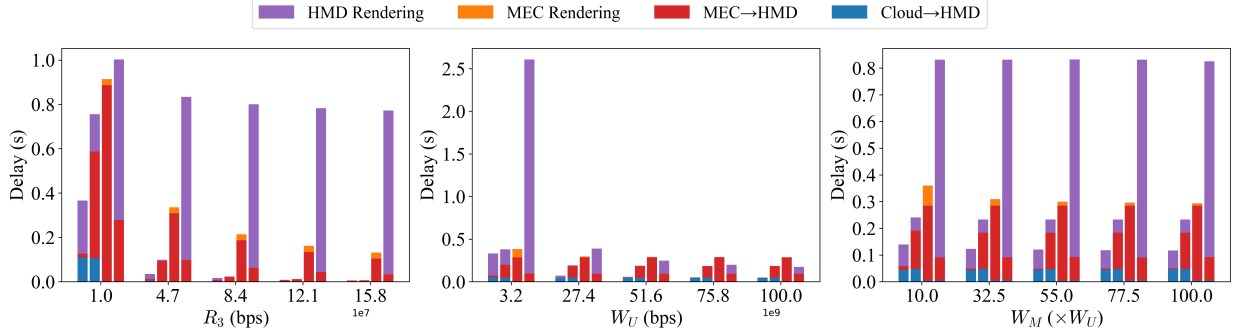


Fig. 11: Delay distribution with respect to R_3 , W_U , and W_M . (Each group of bars corresponds to different schemes: Proposed, Proposed w/o P3CNN, MEC-enabled ($r = 1.0$), and MEC-enabled ($r = 0.0$) in that order.)

This tendency can be observed in the simulation results of both the “P3C (Proposed)” and “MEC-enabled”, however, the “P3C (Proposed)” approach shows a more pronounced reduction in delay as the prediction accuracy increases. When the prediction accuracy increases from 0% to 8%, the delay of the “P3C (Proposed)” decreases by 21.82%, whereas that of the “MEC-enabled” decreases by 12.81%. This result demonstrates that the proposed method achieves more improvement on performance by efficiently coordinating computing and communication processes.

2) *Ablation study*: We conduct an ablation study under the same simulation settings described in Sec.V-A. The system comprises three modules, which are P3C, P3CNN, and parallel processing. The variant “Proposed w/o P3C” excludes the P3C mechanism and utilizes only parallel processing of communication and computation, supplementing the baseline “MEC-enabled”. The “Proposed w/o P3CNN” scheme includes the P3C structure but employs an untrained P3CNN. Lastly, “Proposed w/o parallel” maintains both P3C and P3CNN but assumes a sequential execution between communication and computation. A comparison between “Proposed w/o P3C” and “Proposed w/o P3CNN” shows that incorporating the P3C architecture reduces the average E2E delay by 0.0436 s, corresponding to nearly a 60% improvement. Furthermore, the comparison between “Proposed w/o P3CNN” and “Proposed” yields that the usage of P3CNN yields an additional performance enhancement exceeding 80%. Lastly, the benefit of parallel processing is evident from the 55% reduction in

delay observed when comparing “Proposed w/o Parallel” to the full “Proposed” configuration.

TABLE VI: Average Delay Comparison

Configuration	P3C	P3CNN	Parallel	Avg. Delay (s)
Proposed w/o P3C			✓	0.0740
Proposed w/o P3CNN	✓		✓	0.0314
Proposed w/o parallel	✓	✓		0.0111
Proposed	✓	✓	✓	0.0052

3) *Bottleneck analysis and resource utilization in the P3C*: We conduct an additional experiment and present the corresponding results in two figures. Fig. 10 and 11 illustrate in detail how effectively the proposed P3C framework utilizes system resources. For experimental setup, the range of R_3 is reduced to $[10^{7.2}, 10^{8.2}]$ bps, compared to the value in TABLE. IV for a clearer explanation. Each proactive or delay components are represented as simple stacked bars in the figures, ignoring the parallel execution. In the “MEC-enabled” scheme, the rendering rate r is fixed at $r = 1.0$ or $r = 0.0$ to investigate the performance bottleneck.

Observing “MEC-enabled” in the Fig. 11, we identify the primary bottlenecks of the E2E delay as HMD rendering and MEC→HMD communication. When $r = 1$, the MEC→HMD communication becomes the dominant bottleneck, since most tiles are rendered at the MEC and subsequently transmitted to the HMD, thereby increasing the communication load. Conversely, when r is small, the HMD rendering becomes the main bottleneck because most tiles are transmitted in an

unrendered form, which alleviates the communication burden but imposes a heavy computational load on the HMD. We remark that the data size of an unrendered tile increases with the ratio of h . In Fig. 11, it is clearly observed that the proposed method significantly mitigates the HMD rendering load and the MEC–HMD communication delay. It validates the effect of optimizing the proactive stage in the proposed system. It is interesting to compare the proposed with proposed w/o P3CNN to examine how P3CNN effectively utilizes the resources to reduce the E2E delay and alleviate bottlenecks. We note that the c , r_{pre} , and r are randomly determined with the proposed w/o P3CNN. As R_3 increases in Fig. 10, MEC–HMD transmission of rendered tiles rapidly increases while MEC–HMD transmission of unrendered tiles decreases and HMD rendering also decreases accordingly. It is because the P3CNN increases the number of tiles to be rendered at the MEC as the communication resources become rich. The ratios of delay components rarely changes with the proposed w/o P3CNN, which results in less reduction of E2E delay. Furthermore, as shown in the W_U graph of Fig. 10, as W_U increases, it is observed that the ratio of MEC–HMD transmission of unrendered tiles rapidly increases, whereas the ratio of MEC rendering correspondingly decreases. This behavior occurs because P3CNN adaptively increases the number of tiles rendered at the HMD as the computational resources of the HMD become more abundant. A similar pattern can be observed in the W_M graph of Fig. 10.

VI. CONCLUDING REMARKS

In this work, we propose a novel method for MEC-based low-latency 360° MVRV streaming. The proposed framework, termed P3C, generalizes the proactive stage by integrating computing and communication alongside caching. We establish a comprehensive streaming protocol and develop an analytical framework to quantify the E2E delay under the proposed scheme. Additionally, we design a deep neural network, P3CNN, to optimize the configuration of P3C control variables dynamically based on the streaming environment. Numerical results demonstrate that the proposed approach significantly outperforms conventional methods in terms of delay reduction. For future research, we plan to extend the proposed P3C framework to multi-user scenarios by redesigning both the overall architecture and the P3CNN module to enable optimal allocation of resources. Furthermore, we plan to develop the proposed to be robust to intra-chunk time-varying FoV.

REFERENCES

- [1] Q. Cheng, H. Shan, W. Zhuang, L. Yu, Z. Zhang, and T. Q. Quek, "Design and analysis of mec-and proactive caching-based 360° mobile vr video streaming," *IEEE Transactions on Multimedia*, vol. 24, pp. 1529–1544, 2021.
- [2] B. Soret, P. Mogensen, K. I. Pedersen, and M. C. Aguayo-Torres, "Fundamental tradeoffs among reliability, latency and throughput in cellular networks," in *2014 IEEE Globecom Workshops (GC Wkshps)*. IEEE, 2014, pp. 1391–1396.
- [3] M. Zink, R. Sitaraman, and K. Nahrstedt, "Scalable 360 video stream delivery: Challenges, solutions, and opportunities," *Proceedings of the IEEE*, vol. 107, no. 4, pp. 639–650, 2019.
- [4] X. Ge, L. Pan, Q. Li, G. Mao, and S. Tu, "Multipath cooperative communications networks for augmented and virtual reality transmission," *IEEE Transactions on Multimedia*, vol. 19, no. 10, pp. 2345–2358, 2017.
- [5] J. Park, P. Popovski, and O. Simeone, "Minimizing latency to support vr social interactions over wireless cellular systems via bandwidth allocation," *IEEE Wireless Communications Letters*, vol. 7, no. 5, pp. 776–779, 2018.
- [6] M. S. Elbamby, C. Perfecto, M. Bennis, and K. Doppler, "Toward low-latency and ultra-reliable virtual reality," *IEEE network*, vol. 32, no. 2, pp. 78–84, 2018.
- [7] Y. Chen, J. Teo, J. C. Y. Lai, E. Gunawan, K. S. Low, C. B. Soh, and P. B. Rapajic, "Cooperative communications in ultra-wideband wireless body area networks: Channel modeling and system diversity analysis," *IEEE Journal on Selected Areas in Communications*, vol. 27, no. 1, pp. 5–16, 2009.
- [8] S. Bi, H. Chen, X. Li, S. Wang, Y. Wu, and L. Qian, "A two-stage deep reinforcement learning framework for mec-enabled adaptive 360-degree video streaming," *IEEE Transactions on Mobile Computing*, 2024.
- [9] G. Wu, J. Tang, Y. Zhang, B. Wang, W. Jiang, Z. Xiong, and Y. Feng, "Intelligent edge caching for metaverse applications based on grover search," in *2024 IEEE Annual Congress on Artificial Intelligence of Things (AIoT)*. IEEE, 2024, pp. 194–200.
- [10] T. Lin, Y. Chen, H. Yang, Y. Zhang, B. Jiang, and J. Yan, "Deca360: Deadline-aware edge caching for two-tier 360° video streaming," *Journal of Network and Computer Applications*, vol. 232, p. 104022, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1084804524001991>
- [11] H. Xiao, C. Xu, C. Fang, S. Yang, and L. Zhong, "Vaac-im: Viewing area adaptive control in immersive media transmission," in *Proceedings of the 34th Edition of the Workshop on Network and Operating System Support for Digital Audio and Video*. New York, NY, USA: Association for Computing Machinery, 2024, p. 8–14. [Online]. Available: <https://doi.org/10.1145/3651863.3651877>
- [12] S. Chen, L. Chen, Y. Tang, and W. Wang, "Collaborative edge caching and dynamic bitrate adaptation for shvc-based vr video streaming," in *2024 IEEE 49th Conference on Local Computer Networks (LCN)*. IEEE, 2024, pp. 1–9.
- [13] M. Xu, Y. Song, J. Wang, M. Qiao, L. Huo, and Z. Wang, "Predicting head movement in panoramic video: A deep reinforcement learning approach," *IEEE transactions on pattern analysis and machine intelligence*, vol. 41, no. 11, pp. 2693–2708, 2018.
- [14] M. Hosseini and V. Swaminathan, "Adaptive 360 vr video streaming: Divide and conquer," in *2016 IEEE International Symposium on Multimedia (ISM)*. IEEE, 2016, pp. 107–110.
- [15] Y. Sun, Z. Chen, M. Tao, and H. Liu, "Communications, caching, and computing for mobile virtual reality: Modeling and tradeoff," *IEEE Transactions on Communications*, vol. 67, no. 11, pp. 7573–7586, 2019.
- [16] J. Chakareski, "Vr/ar immersive communication: Caching, edge computing, and transmission trade-offs," in *Proceedings of the Workshop on Virtual Reality and Augmented Reality Network*, 2017, pp. 36–41.
- [17] X. Yang, Z. Chen, K. Li, Y. Sun, N. Liu, W. Xie, and Y. Zhao, "Communication-constrained mobile edge computing systems for wireless virtual reality: Scheduling and tradeoff," *IEEE Access*, vol. 6, pp. 16665–16677, 2018.
- [18] T. Stockhammer, "Dynamic adaptive streaming over http– standards and design principles," in *Proceedings of the second annual ACM conference on Multimedia systems*, 2011, pp. 133–144.
- [19] Y. Jin, J. Liu, F. Wang, and S. Cui, "Ebublio: Edge-assisted multiuser 360 video streaming," *IEEE Internet of Things Journal*, vol. 10, no. 17, pp. 15408–15419, 2023.
- [20] S. Kumar, L. Bhagat, J. Jin *et al.*, "Multi-neural network based tiled 360° video caching with mobile edge computing," *Journal of Network and Computer Applications*, vol. 201, p. 103342, 2022.
- [21] J. Yang, Z. Guo, J. Luo, Y. Shen, and K. Yu, "Cloud-edge-end collaborative caching based on graph learning for cyber-physical virtual reality," *IEEE systems journal*, vol. 17, no. 4, pp. 5097–5108, 2023.
- [22] J. Ren, G. Yu, Y. Cai, and Y. He, "Latency optimization for resource allocation in mobile-edge computation offloading," *IEEE Transactions on Wireless Communications*, vol. 17, no. 8, pp. 5506–5519, 2018.
- [23] H. A. Alameddine, S. Sharafeddine, S. Sebbah, S. Ayoubi, and C. Assi, "Dynamic task offloading and scheduling for low-latency iot services in multi-access edge computing," *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 3, pp. 668–682, 2019.
- [24] L. Bazzani, H. Larochelle, and L. Torresani, "Recurrent mixture density network for spatiotemporal visual attention," *arXiv preprint arXiv:1603.08199*, 2016.
- [25] M. A. Khan, E. Baccour, Z. Chkribene, A. Erbad, R. Hamila, M. Hamdi, and M. Gabbouj, "A survey on mobile edge computing for video streaming: Opportunities and challenges," *IEEE Access*, vol. 10, pp. 120514–120550, 2022.

- [26] B. Hayes, Y. Chang, and G. Riley, "Controlled unfair adaptive 360 vr video delivery over an mptcp/quic architecture," in *2018 IEEE International Conference on Communications (ICC)*, 2018, pp. 1–6.
- [27] C. Yin, Z. Chen, Y. Hu, and K. Yu, "Fine-grained transmission optimization of large-scale web vr scenes," in *2018 IEEE International Conference on Progress in Informatics and Computing (PIC)*, 2018, pp. 209–214.
- [28] C. Zhou, C.-W. Lin, and Z. Guo, "mdash: A markov decision-based rate adaptation approach for dynamic http streaming," *IEEE Transactions on Multimedia*, vol. 18, no. 4, pp. 738–751, 2016.
- [29] Z. Pang, "Vatp360: Viewport adaptive 360-degree video streaming based on tile priority," *arXiv preprint arXiv:2307.15984*, 2023.
- [30] C. Madarasingha and K. Thilakarathna, "Vastile: Viewport adaptive scalable 360-degree video frame tiling," in *Proceedings of the 29th ACM International Conference on Multimedia*, 2021, pp. 4555–4563.
- [31] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 4510–4520.
- [32] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [33] M. Shoenybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-lm: Training multi-billion parameter language models using model parallelism," *arXiv preprint arXiv:1909.08053*, 2019.
- [34] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever *et al.*, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.
- [35] X. Corbillon, G. Simon, A. Devlic, and J. Chakareski, "Viewport-adaptive navigable 360-degree video delivery," in *2017 IEEE international conference on communications (ICC)*. IEEE, 2017, pp. 1–7.



S. Park received her B.S. degree in information and communication engineering from Myongji University in Yongin, Gyeonggi-do, South Korea, in 2024. Her research interests lie in the field of computer vision, large language model, and reinforcement learning.



M. Kim received his B.S. degree in computer engineering from Myongji University in Yongin, Gyeonggi-do, South Korea, in 2025. His research interests lie in the field of computer vision, large language model, and federated learning.



Y. Ma received her B.S. degree in mathematics from Myongji University in Yongin, Gyeonggi-do, South Korea, in 2024. Her research interests lie in the field of computer vision, large language model, and federated learning.



Dong H. Kim (SM'13) received his B.S. from Yonsei University in 1997 and M.S. and Ph.D. degrees in Electrical Engineering from Korea Advanced Institute of Science and Technology (KAIST), Republic of Korea, in 1999 and 2004 respectively. From 2004 to 2006, he was Senior Member of Technical Staff, 4G Wireless Technology Lab., Samsung Advanced Institute of Technology (SAIT). From 2004 to 2006 he was Senior Researcher, Mobile Communications Research Institute, Samsung Electronics. Since 2007, he has been Professor with the Seoul National University of Science and Technology. His current research interests include the design of immersive media(VR/AR) and 5G/6G communication systems.



C. You (S'93) received the B.S., M.S., and Ph.D. degrees in electronics engineering from Yonsei University, Seoul, Republic of Korea, in 1993, 1995, and 1999, respectively. From Jan. 1999 to April 2003, he worked as a Senior Research Engineer with the LG Electronics, Gyeonggi, Korea. During 2003–2004, he was a Senior Research Engineer at the EoNex, Songnam, Korea. From August 2004 to July 2006, he was with the Samsung Electronics, Suwon, Korea. Since September 2006, he has been with the Department of Information and Communications Engineering, Myongji University, Gyeonggi, Yongin, Republic of Korea. His research areas are next generation communication systems, artificial intelligence, air I/F technologies in the international standards, communication theory, and signal processing. He is currently interested in the 5G/6G communication systems, machine & deep learning, AR/VR, IoT/AIoT, and V2X.



J.-H. Ahn received his B.S. and M.S. degree in mathematics in 2013 and 2016, and Ph.D. degree in electrical engineering in 2020, all from Korea Advanced Institute of Science and Technology (KAIST), Daejeon, Korea. He is currently working as an assistant professor at the Department of Information and Communication Engineering, Myongji University, Yongin, Gyeonggi-do, South Korea. His research interests lie in the field of probability theory, communication theory, and machine learning, with a specific focus on federated learning. From January to September 2019, he was a visiting researcher at King's Communications, Learning and Information Processing lab, King's College London, United Kingdom. He worked as a research associate in KAIST from September 2019 to September 2020. He was a research associate in the MGH/BWH Center for Advanced Medical Computing and Analysis, department of radiology, Massachusetts General Hospital and Harvard Medical School from October 2021 to August 2022.